[Year]

PHONE INVENTORY MANAGEMENT SYSTEM

UGC-0423044

[COMPANY NAME] | [Company address]

PHONE INVENTORY MANAGEMENT SYSTEM

Database Design and Query Optimization Using PostgreSQL

---

1. Introduction

The Phone Inventory Management System is designed to manage mobile phone inventory, suppliers, warehouses, customers, employees, purchases, and sales efficiently. The system helps businesses track stock levels, monitor sales transactions, manage supplier purchases, and generate operational reports.

The database was implemented using PostgreSQL and optimized using indexing techniques to improve query performance when handling large volumes of data.

The main objective of this project is to:

- Design a normalized relational database
- Generate large amounts of dummy data using loops
- Identify frequently used queries
- Analyze query performance
- Apply indexing techniques
- Compare performance before and after indexing

---

2. Objectives

- To create a relational database for a Phone Inventory Management System
- To generate approximately 10,000 records for major entities
- To analyze frequently executed queries
- To improve query performance using indexes
- To compare query execution times before and after indexing
- To understand the importance of database tuning and optimization

3. Database Management System

Database System: PostgreSQL

Tools Used:

- PostgreSQL
- pgAdmin
- SQL Shell (psql)

---

4. Database Design

4.1 Entities

The system consists of the following entities:

1. Brand
2. PhoneModel
3. Supplier
4. Warehouse
5. Customer
6. Employee
7. Inventory
8. PurchaseOrder
9. PurchaseOrderItem
10. Sale

4.2 Entity Relationships

Brand → PhoneModel

One brand can have many phone models.

Warehouse → Inventory

One warehouse can store many phone models.

PhoneModel → Inventory

One phone model can be stored in multiple warehouses.

Supplier → PurchaseOrder

One supplier can supply many purchase orders.

Employee → PurchaseOrder

One employee can create many purchase orders.

PurchaseOrder → PurchaseOrderItem

One purchase order can contain many items.

PhoneModel → PurchaseOrderItem

One phone model can appear in many purchase order items.

Customer → Sale

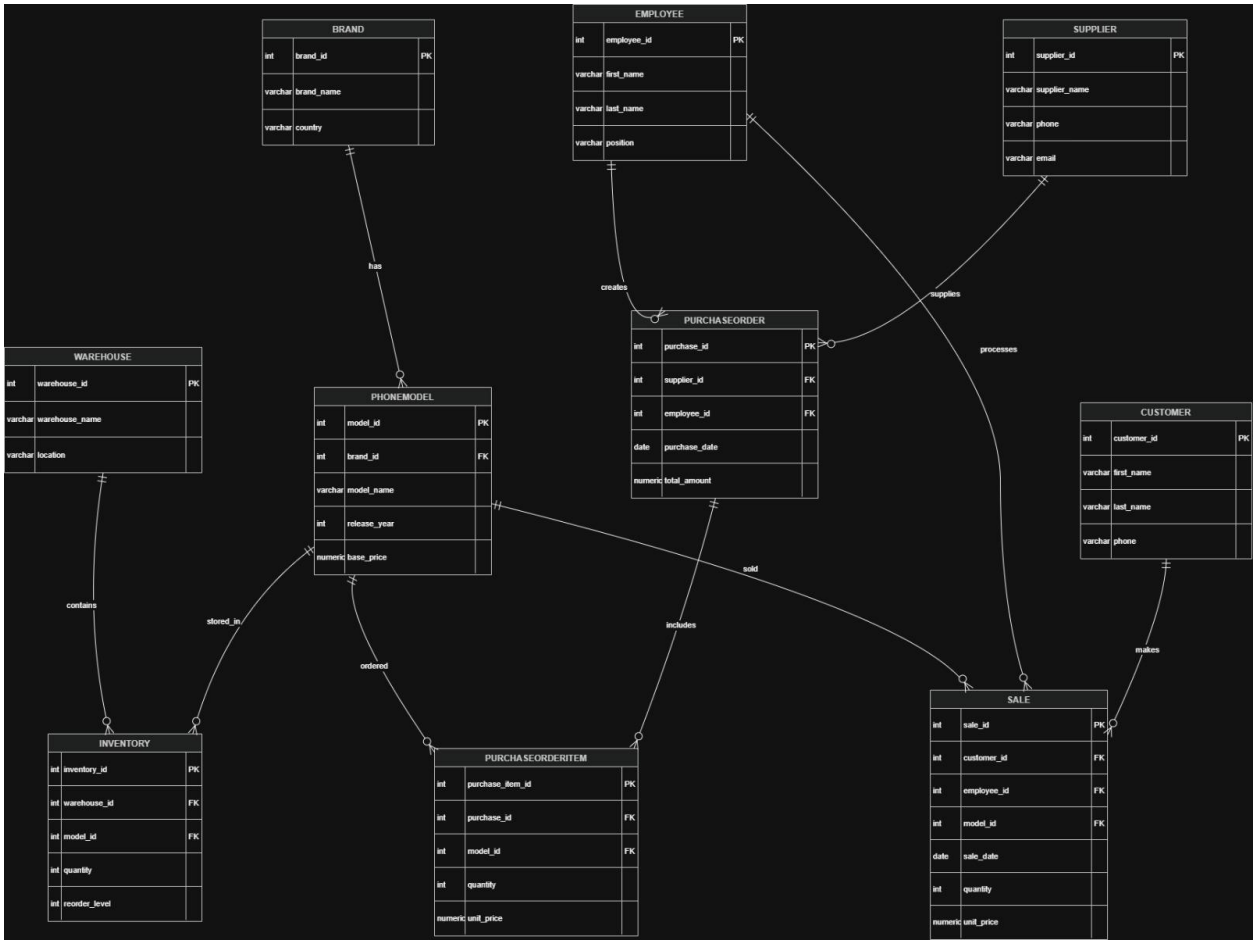
One customer can make many purchases.

Employee → Sale

One employee can process many sales.

PhoneModel → Sale

One phone model can be sold many times


BRAND
int brand_id PK
varchar brand_name
varchar country
EMPLOYEE
int employee_id PK
varchar first_name
varchar last_name
varchar position
SUPPLIER
int supplier_id PK
varchar supplier_name
varchar phone
varchar email
has
creates
supplies
processes
WAREHOUSE
int warehouse_id PK
varchar warehouse_name
varchar location
PHONEMODEL
int model_id PK
int brand_id FK
varchar model_name
int release_year
numeric base_price
PURCHASEORDER
int purchase_id PK
int supplier_id FK
int employee_id FK
date purchase_date
numeric total_amount
CUSTOMER
int customer_id PK
varchar first_name
varchar last_name
varchar phone
contains
stored_in
ordered
includes
sold
makes
INVENTORY
int inventory_id PK
int warehouse_id FK
int model_id FK
int quantity
int reorder_level
PURCHASEORDERITEM
int purchase_item_id PK
int purchase_id FK
int model_id FK
int quantity
numeric unit_price
SALE
int sale_id PK
int customer_id FK
int employee_id FK
int model_id FK
date sale_date
int quantity
numeric unit_price

5. Table Structure

1. Brand

```
CREATE TABLE Brand (
   brand_id SERIAL PRIMARY KEY,
   brand_name VARCHAR(100) NOT NULL,
   country VARCHAR(100)
);
```

---

2. PhoneModel

```
CREATE TABLE PhoneModel (
   model_id SERIAL PRIMARY KEY,
   brand_id INT REFERENCES Brand(brand_id),
   model_name VARCHAR(100),
   release_year INT,
   base_price NUMERIC(10,2)
);
```

---

3. Supplier

```
CREATE TABLE Supplier (
   supplier_id SERIAL PRIMARY KEY,
   supplier_name VARCHAR(100),
   phone VARCHAR(20),
   email VARCHAR(100),
   address TEXT
);
```

---

4. Warehouse

```
CREATE TABLE Warehouse (
   warehouse_id SERIAL PRIMARY KEY,
   warehouse_name VARCHAR(100),
   location VARCHAR(100)
);
```

---

5. Customer

```
CREATE TABLE Customer (
    customer_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    phone VARCHAR(20),
    email VARCHAR(100)
);
```

---

6. Employee

```
CREATE TABLE Employee (
    employee_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    position VARCHAR(50),
    salary NUMERIC(10,2),
    hire_date DATE
);
```

---

7. Inventory

```
CREATE TABLE Inventory (
    inventory_id SERIAL PRIMARY KEY,
    warehouse_id INT REFERENCES Warehouse(warehouse_id),
    model_id INT REFERENCES PhoneModel(model_id),
    quantity INT,
    reorder_level INT
);
```

---

8. PurchaseOrder

```
CREATE TABLE PurchaseOrder (
    purchase_id SERIAL PRIMARY KEY,
    supplier_id INT REFERENCES Supplier(supplier_id),
    employee_id INT REFERENCES Employee(employee_id),
    purchase_date DATE,
    total_amount NUMERIC(12,2)
);
```

---

9. PurchaseOrderItem

```
CREATE TABLE PurchaseOrderItem (
    purchase_item_id SERIAL PRIMARY KEY,
    purchase_id INT REFERENCES PurchaseOrder(purchase_id),
    model_id INT REFERENCES PhoneModel(model_id),
```

```
    quantity INT,
    unit_price NUMERIC(10,2)
);
```

---

10. Sale

```
CREATE TABLE Sale (
    sale_id SERIAL PRIMARY KEY,
    customer_id INT REFERENCES Customer(customer_id),
    employee_id INT REFERENCES Employee(employee_id),
    model_id INT REFERENCES PhoneModel(model_id),
    sale_date DATE,
    quantity INT,
    unit_price NUMERIC(10,2)
);
```

6. Dummy Data Generation

To simulate a real-world environment, large amounts of data were generated using PostgreSQL loops and DO blocks.

Example Dummy Data Generation

```
DO $$

DECLARE

    i INT;

BEGIN

    FOR i IN 1..10000 LOOP

        INSERT INTO Customer(

            first_name,

            last_name,

            phone,

            email

        )

        VALUES(

            'CustomerF_'||i,
```

```
        'CustomerL_'||i,
        '07'||LPAD(i::TEXT,8,'0'),
        'customer'||i||'@gmail.com'
    );
  END LOOP;
END $$;
```

7.Frequently Used Queries and Indexing Strategy

Query 1 – Customer Purchase History

Purpose

Customer service staff frequently check the purchase history of customers for warranty claims and support requests.

Query

```
SELECT *
FROM Sale
WHERE customer_id = 5000;
```

Index Creation

```
CREATE INDEX idx_sale_customer
ON Sale(customer_id);
```

```
phone_inventory_db=# EXPLAIN ANALYZE
phone_inventory_db-# SELECT *
phone_inventory_db-# FROM Sale
phone_inventory_db-# WHERE customer_id = 5000;
                                                 QUERY PLAN
-------------------------------------------------------------------------------------------- Seq Scan on sale  (cost=0
.00..993.00 rows=15 width=30) (actual time=0.369..3.179 rows=10.00 loops=1)
   Filter: (customer_id = 5000)
   Rows Removed by Filter: 49990
   Buffers: shared hit=368
 Planning Time: 0.146 ms
 Execution Time: 3.204 ms
(6 rows)


phone_inventory_db=# CREATE INDEX idx_sale_customer
phone_inventory_db-# ON Sale(customer_id);
CREATE INDEX
phone_inventory_db=# EXPLAIN ANALYZE
phone_inventory_db-# SELECT *
phone_inventory_db-# FROM Sale
phone_inventory_db-# WHERE customer_id = 5000;
                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
------------
 Bitmap Heap Scan on sale  (cost=4.41..55.51 rows=15 width=30) (actual time=0.105..0.115 rows=10.00 loops=1)
   Recheck Cond: (customer_id = 5000)
   Heap Blocks: exact=10
   Buffers: shared hit=10 read=2
   ->  Bitmap Index Scan on idx_sale_customer  (cost=0.00..4.40 rows=15 width=0) (actual time=0.094..0.094 rows=10.
00 loops=1)
         Index Cond: (customer_id = 5000)
         Index Searches: 1
         Buffers: shared read=2
 Planning:
   Buffers: shared hit=15 read=1
 Planning Time: 0.670 ms
 Execution Time: 0.129 ms
(12 rows)


phone_inventory_db=#
```

Query 2 – Sales by Phone Model

Purpose

Managers frequently analyze sales performance of specific phone models.

Query

```
SELECT *
FROM Sale
WHERE model_id = 2000;
```

Index Creation

```
CREATE INDEX idx_sale_model
ON Sale(model_id);
```

```
phone_inventory_db=# EXPLAIN ANALYZE
phone_inventory_db-# SELECT *
phone_inventory_db-# FROM Sale
phone_inventory_db-# WHERE model_id = 2000;
                                          QUERY PLAN
-----------------------------------------------------------------------------------------------
 Seq Scan on sale  (cost=0.00..993.00 rows=5 width=30) (actual time=0.228..2.121 rows=5.00 loops=1)
   Filter: (model_id = 2000)
   Rows Removed by Filter: 49995
   Buffers: shared hit=368
 Planning Time: 0.092 ms
 Execution Time: 2.140 ms
(6 rows)


phone_inventory_db=# CREATE INDEX idx_sale_model
phone_inventory_db-# ON Sale(model_id);
CREATE INDEX
phone_inventory_db=# EXPLAIN ANALYZE
phone_inventory_db-# SELECT *
phone_inventory_db-# FROM Sale
phone_inventory_db-# WHERE model_id = 2000;
                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on sale  (cost=4.33..22.64 rows=5 width=30) (actual time=0.037..0.041 rows=5.00 loops=1)
   Recheck Cond: (model_id = 2000)
   Heap Blocks: exact=5
   Buffers: shared hit=5 read=2
   ->  Bitmap Index Scan on idx_sale_model  (cost=0.00..4.33 rows=5 width=0) (actual time=0.025..0.026 rows=5.00 lo
ops=1)
         Index Cond: (model_id = 2000)
         Index Searches: 1
         Buffers: shared read=2
 Planning:
   Buffers: shared hit=15 read=1 dirtied=2
 Planning Time: 0.911 ms
 Execution Time: 0.056 ms
(12 rows)


phone_inventory_db=#
```

Query 3 – Inventory Availability Check

Purpose

Employees regularly check stock levels before completing customer sales.

Query

```
SELECT *
FROM Inventory
WHERE warehouse_id = 50
AND model_id = 500;
```

Index Creation

```
CREATE INDEX idx_inventory_wh_model
ON Inventory(warehouse_id, model_id);
```

```
phone_inventory_db=# EXPLAIN ANALYZE
phone_inventory_db-# SELECT *
phone_inventory_db-# FROM Inventory
phone_inventory_db-# WHERE warehouse_id = 50
phone_inventory_db-# AND model_id = 500;
                                          QUERY PLAN
-----------------------------------------------------------------------------------------------
 Seq Scan on inventory  (cost=0.00..214.00 rows=1 width=20) (actual time=0.611..0.611 rows=0.00 loops=1)
   Filter: ((warehouse_id = 50) AND (model_id = 500))
   Rows Removed by Filter: 10000
   Buffers: shared hit=64
 Planning Time: 0.231 ms
 Execution Time: 0.636 ms
(6 rows)


phone_inventory_db=# CREATE INDEX idx_inventory_wh_model
phone_inventory_db-# ON Inventory(warehouse_id, model_id);
CREATE INDEX
phone_inventory_db=# EXPLAIN ANALYZE
phone_inventory_db-# SELECT *
phone_inventory_db-# FROM Inventory
phone_inventory_db-# WHERE warehouse_id = 50
phone_inventory_db-# AND model_id = 500;
                                                    QUERY PLAN

-------------------------------------------------------------------------------------------------------------------
------------------
 Index Scan using idx_inventory_wh_model on inventory  (cost=0.29..8.30 rows=1 width=20) (actual time=0.080..0.080
rows=0.00 loops=1)
   Index Cond: ((warehouse_id = 50) AND (model_id = 500))
   Index Searches: 1
   Buffers: shared read=2
 Planning:
   Buffers: shared hit=18 read=1
 Planning Time: 1.300 ms
 Execution Time: 0.096 ms
(8 rows)


phone_inventory_db=#
```

Query 4 – Purchase Orders by Supplier

Purpose

Procurement officers review orders placed with suppliers.

Query

```
SELECT *
FROM PurchaseOrder
WHERE supplier_id = 100;
```

Index Creation

```
CREATE INDEX idx_purchase_supplier
ON PurchaseOrder(supplier_id);
```

```
phone_inventory_db=# EXPLAIN ANALYZE
phone_inventory_db-# SELECT *
phone_inventory_db-# FROM PurchaseOrder
phone_inventory_db-# WHERE supplier_id = 100;
                                                 QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Seq Scan on purchaseorder  (cost=0.00..189.00 rows=18 width=22) (actual time=0.047..0.913 rows=19.00 loops=1)
   Filter: (supplier_id = 100)
   Rows Removed by Filter: 9981
   Buffers: shared hit=64
 Planning Time: 0.099 ms
 Execution Time: 0.935 ms
(6 rows)


phone_inventory_db=# CREATE INDEX idx_purchase_supplier
phone_inventory_db-# ON PurchaseOrder(supplier_id);
CREATE INDEX
phone_inventory_db=# EXPLAIN ANALYZE
phone_inventory_db-# SELECT *
phone_inventory_db-# FROM PurchaseOrder
phone_inventory_db-# WHERE supplier_id = 100;
                                                           QUERY PLAN

-------------------------------------------------------------------------------------------------------------
----------------
 Bitmap Heap Scan on purchaseorder  (cost=4.42..44.65 rows=18 width=22) (actual time=0.081..0.113 rows=19.00 loops=
1)
   Recheck Cond: (supplier_id = 100)
   Heap Blocks: exact=17
   Buffers: shared hit=17 read=2
   ->  Bitmap Index Scan on idx_purchase_supplier  (cost=0.00..4.42 rows=18 width=0) (actual time=0.056..0.057 rows
=19.00 loops=1)
         Index Cond: (supplier_id = 100)
         Index Searches: 1
         Buffers: shared read=2
 Planning:
   Buffers: shared hit=15 read=1
 Planning Time: 1.520 ms
 Execution Time: 0.140 ms
(12 rows)


phone_inventory_db=#
```

Query 5 – Sales Report by Date Range

Purpose

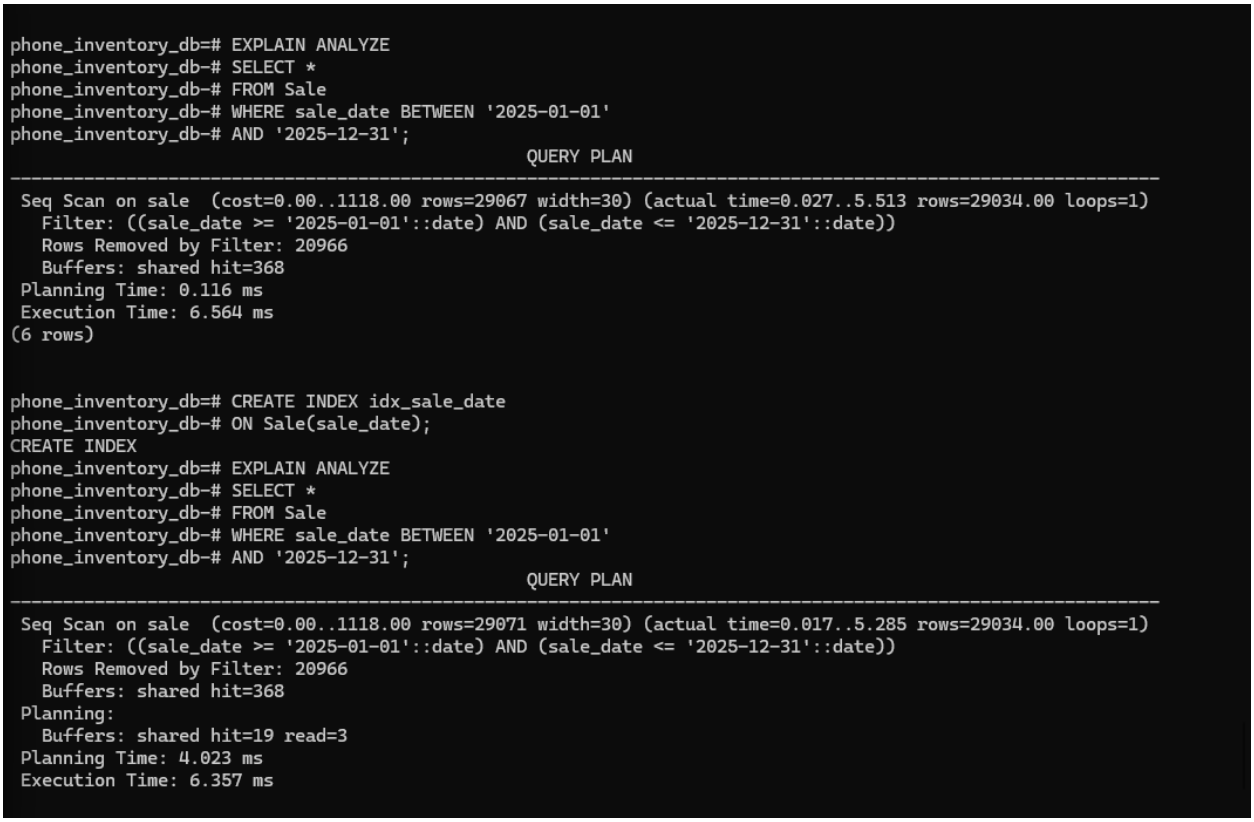
Managers generate monthly and yearly sales reports.

Query

```
SELECT *
FROM Sale
WHERE sale_date BETWEEN '2025-01-01'
AND '2025-12-31';
```

Index Creation

```
CREATE INDEX idx_sale_date
ON Sale(sale_date);
```



```
phone_inventory_db=# EXPLAIN ANALYZE
phone_inventory_db-# SELECT *
phone_inventory_db-# FROM Sale
phone_inventory_db-# WHERE sale_date BETWEEN '2025-01-01'
phone_inventory_db-# AND '2025-12-31';
                                                 QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Seq Scan on sale  (cost=0.00..1118.00 rows=29067 width=30) (actual time=0.027..5.513 rows=29034.00 loops=1)
   Filter: ((sale_date >= '2025-01-01'::date) AND (sale_date <= '2025-12-31'::date))
   Rows Removed by Filter: 20966
   Buffers: shared hit=368
 Planning Time: 0.116 ms
 Execution Time: 6.564 ms
(6 rows)


phone_inventory_db=# CREATE INDEX idx_sale_date
phone_inventory_db-# ON Sale(sale_date);
CREATE INDEX
phone_inventory_db=# EXPLAIN ANALYZE
phone_inventory_db-# SELECT *
phone_inventory_db-# FROM Sale
phone_inventory_db-# WHERE sale_date BETWEEN '2025-01-01'
phone_inventory_db-# AND '2025-12-31';
                                                 QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Seq Scan on sale  (cost=0.00..1118.00 rows=29071 width=30) (actual time=0.017..5.285 rows=29034.00 loops=1)
   Filter: ((sale_date >= '2025-01-01'::date) AND (sale_date <= '2025-12-31'::date))
   Rows Removed by Filter: 20966
   Buffers: shared hit=368
 Planning:
   Buffers: shared hit=19 read=3
 Planning Time: 4.023 ms
 Execution Time: 6.357 ms
```

10. Query Planner Analysis

Before indexing, PostgreSQL typically used Sequential Scan operations to search through large tables. Sequential scanning requires examining every row, which increases execution time as table size grows.

After creating indexes, PostgreSQL utilized Index Scan operations. Index scans allow direct access to matching records without reading the entire table.

As a result:

- Query execution time decreased
- Database responsiveness improved
- CPU and I/O usage were reduced
- Large datasets became easier to manage

---

11. Conclusion

- The Phone Inventory Management System database was successfully designed and implemented using PostgreSQL. The system includes ten interconnected entities that support inventory management, supplier management, purchase processing, and sales tracking.

- Large datasets containing up to 50,000 records were generated using PostgreSQL loops to simulate a realistic business environment.
- Frequently executed queries were identified and analyzed using EXPLAIN ANALYZE. Appropriate indexes were then created on commonly searched columns. The comparison of query performance before and after indexing demonstrated significant improvements in execution speed and overall database efficiency.
- This project highlights the importance of indexing and query optimization techniques in improving the performance of large-scale database systems.

---

12. References

1. PostgreSQL Documentation
2. PostgreSQL SQL Shell (psql)
3. PostgreSQL Query Optimization Guide
4. Database Management Systems Lecture Notes
5. PostgreSQL Indexing Documentation